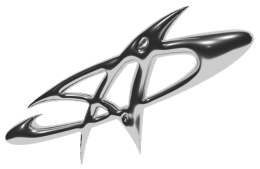
Week 01 · Orientation & Deep Learning Basics

인공지능은 어떻게 학습하는가

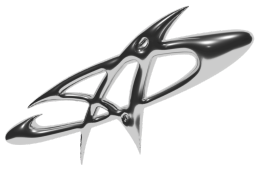
딥러닝이 손실을 줄여가는 원리를 복습하고, 앞으로 12주 동안 Hugging Face 없이 Transformer를 직접 손으로 구현해 나갈 로드맵을 확인합니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

이론 세션

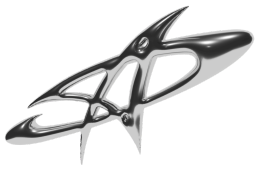
왜 처음부터 Transformer를 직접 만드는가



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

이 강의는 13주차까지 Hugging Face의 `Trainer` 없이,
모든 학습 루프를 우리 손으로 for-loop로 직접 짭니다.

Hugging Face는 12주차에 딱 한 번, "이게 실무에서 어떻게 대체되는지" 보여드리기 위해서만 등장합니다.

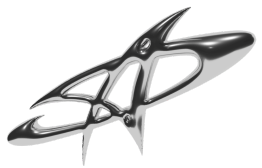


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

13주 로드맵 한눈에 보기

우리는 4개의 큰 구간을 지나갑니다

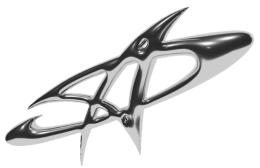
- **1~2주차 · 기초 복습** — 딥러닝 학습 원리, RNN/LSTM은 아주 간단히만
- **3~7주차 · Transformer 블록 직접 구현** — Attention → Multi-Head → Encoder/Decoder → 전체 조립
- **8~11주차 · 실전 학습 & 확장** — 한국어 챗봇 데이터로 학습, 디코딩 전략, GPT식 Decoder-only
- **12~13주차 · 실무 연결** — Hugging Face 비교, Streamlit으로 나만의 챗봇 Agent MVP



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

학습이란, 예측값을 정답에 가깝게 맞춰가는 과정

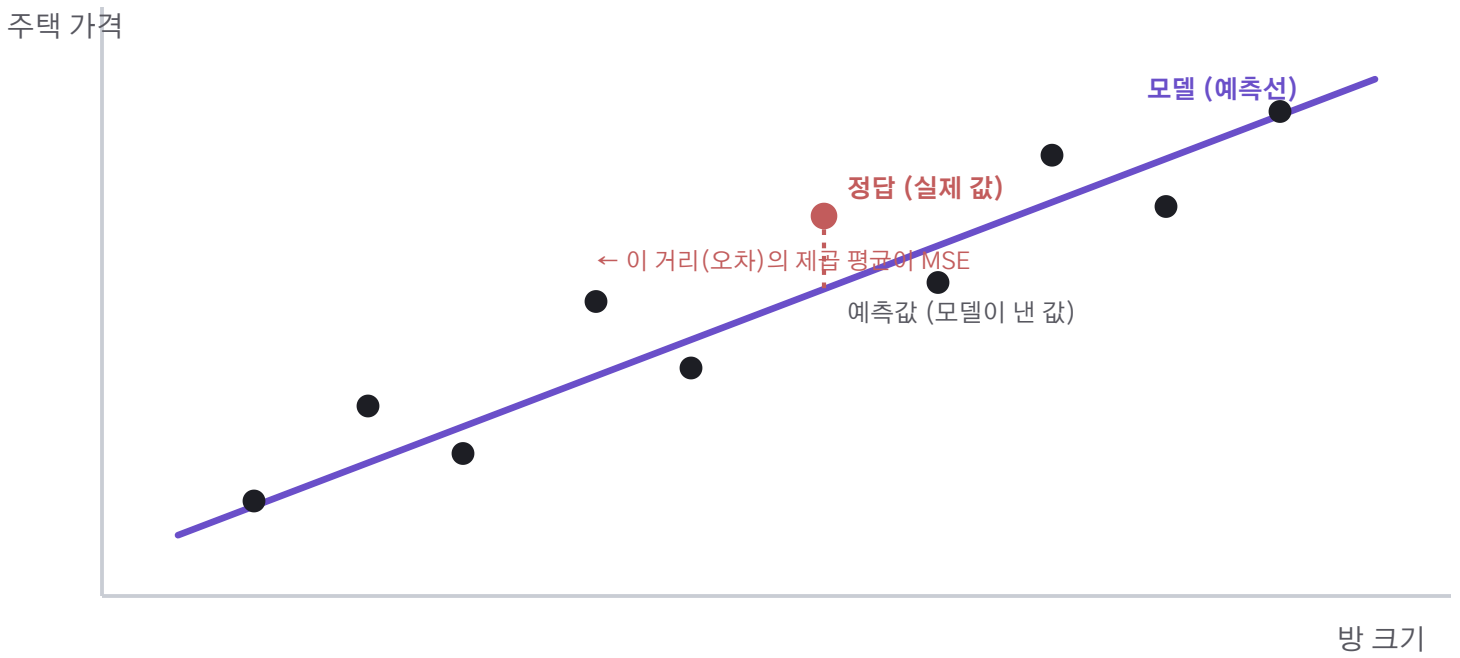
- 모델은 입력을 받아 **예측값(prediction)**을 출력한다
- 예측값과 **정답(target)**을 비교해 얼마나 틀렸는지 점수를 매긴다 — 이 점수가 **손실(Loss)**
- 그 손실이 **작아지는 방향**으로 모델 내부의 파라미터(가중치)를 아주 조금씩 바꾼다
- 이 과정을 데이터 전체에 대해 수천~수만 번 반복하는 것이 “학습”이다



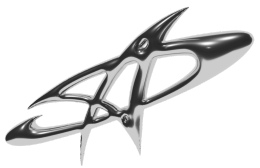
이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

손실을 눈으로 보면

점들과 선 사이의 “거리”가 곧 손실이다



점(실제 데이터)과 모델이 그은 선(예측) 사이의 세로 거리가 클수록, 그 샘플의 손실이 크다는 뜻입니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

손실 함수 (1) — 회귀 문제

MSE (Mean Squared Error)

집값 예측처럼 “숫자”를 맞히는 회귀(regression) 문제에서 가장 기본이 되는 손실 함수입니다.

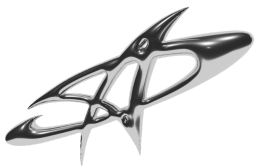
$$L = \frac{1}{n} \Sigma_{i=1..n} (y_i - \hat{y}_i)^2$$

y_i — i번째 샘플의 실제 정답값

$\hat{y}_i$ — i번째 샘플에 대한 모델의 예측값

n — 샘플(데이터) 개수

제곱 — 오차의 부호를 없애고, 크게 틀릴수록 더 크게 벌점을 준다



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

손실 함수 (2) — 분류 문제

Cross Entropy Loss

Transformer가 다음 단어를 “맞히는” 것도 결국 “수만 개의 단어 중 정답 클래스 고르기”라는 분류(classification) 문제입니다.

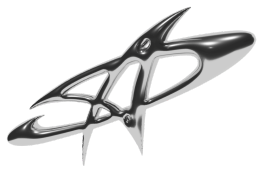
$$L = -\Sigma_c \, y_c \log(\hat{y}_c)$$

c — 정답 클래스(예: 정답 단어)의 인덱스

y_c — 정답이면 1, 아니면 0 (one-hot)

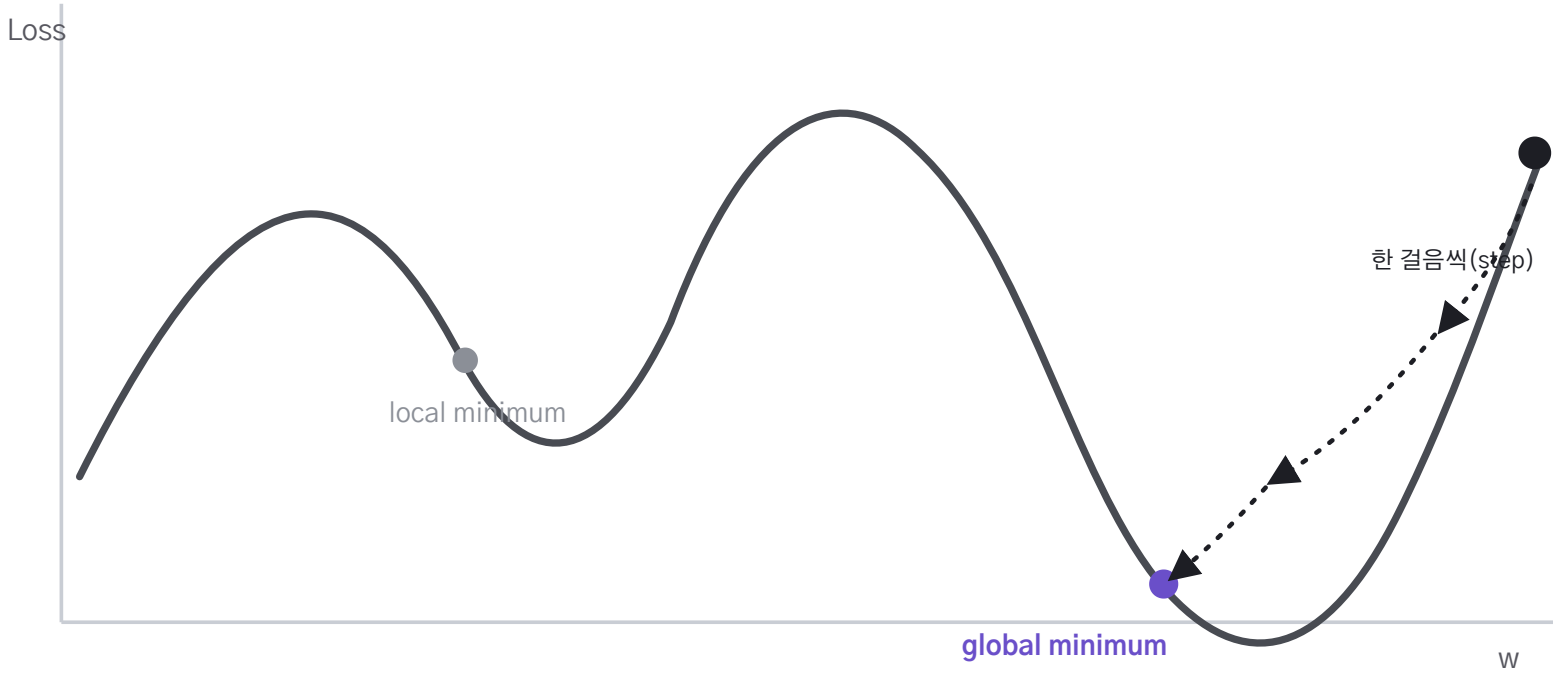
ŷ_c — 모델이 클래스 c일 확률로 예측한 값 (softmax 출력)

의미 — 정답에 낮은 확률을 줄수록 손실이 기하급수적으로 커진다



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

경사하강법 (Gradient Descent)



기울기(gradient)가 가장 가파르게 내려가는 방향으로 조금씩 이동한다 — 잘못하면 얕은 local minimum에 갇힐 수도 있다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

경사하강법의 수식

파라미터 업데이트 규칙

“기울기(gradient)의 반대 방향으로, 학습률만큼 이동한다”는 규칙을 그대로 수식으로 옮긴 것뿐입니다.

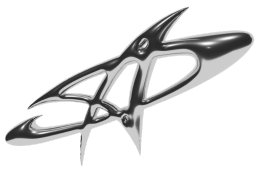
$$w \leftarrow w - \eta \cdot \nabla_w L(w)$$

w — 모델의 파라미터(가중치) 하나

$\nabla_w L(w)$ — w에 대한 손실의 기울기 — “w를 늘리면 손실이 커지는지 작아지는지”

η (eta) — 학습률(learning rate) — 한 걸음의 보폭

PyTorch에서는 — `loss.backward()`가 ∇를 계산, `optimizer.step()`이 이 업데이트를 수행

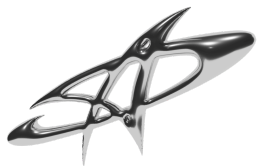


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

옵티마이저 (Optimizer)

“보폭”과 “방향”을 더 똑똑하게 정하는 방법들

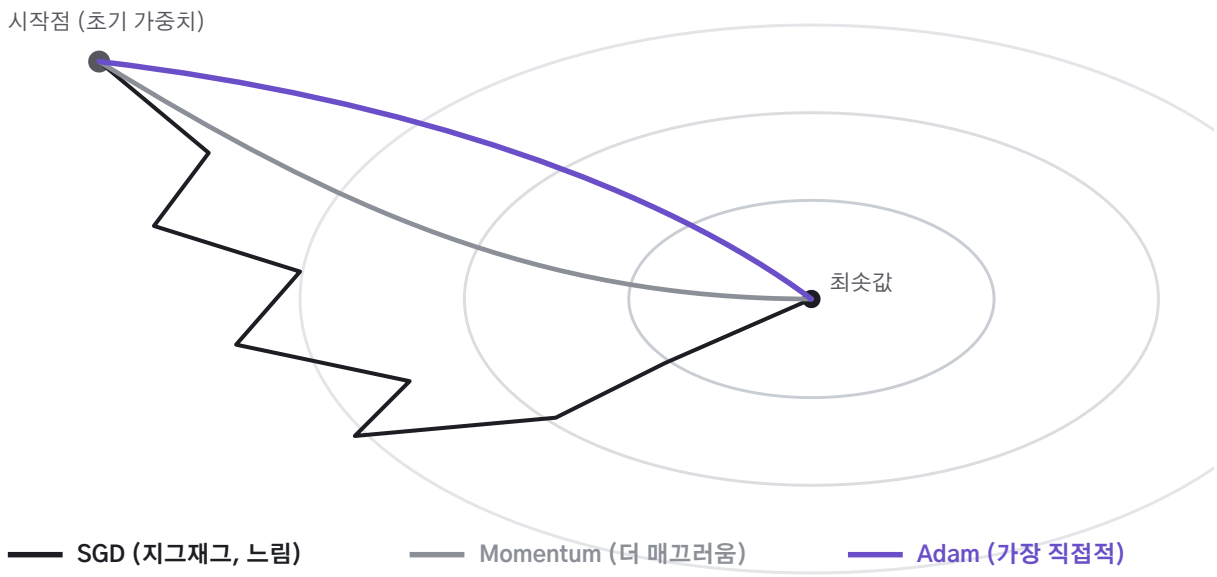
- **SGD** — 매 스텝마다 현재 기울기 방향으로만 이동. 단순하지만 지그재그로 흔들릴 수 있음
- **Momentum** — 이전 이동 방향을 “관성”처럼 누적해서 흔들림을 줄이고 더 빠르게 수렴
- **Adam** — 파라미터마다 학습률을 자동으로 조절(적응형). 현재 딥러닝에서 가장 널리 쓰이는 기본값
- 이 강의에서는 특별한 언급이 없으면 기본적으로 **Adam(또는 AdamW)**을 사용합니다



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

옵티마이저마다 다른 경로

같은 출발점, 다른 길로 최솟값을 찾아간다



등고선은 손실 지형(loss landscape)을 뜻합니다 — SGD는 매 걸음을 그대로 따라가 흔들리고, Momentum/Adam은 관성·적응형 보폭으로 더 매끄럽게 수렴합니다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

학습률(learning rate) 감각 잡기

η(학습률)이 너무 크면? 너무 작으면?

너무 크다 (e.g. 1.0)

- 한 걸음이 너무 커서 최솟값을 계속 “지나쳐” 버림
- loss가 줄지 않고 오히려 튀거나 발산(NaN)
- 체감: 학습이 불안정하고 그래프가 요동친다

너무 작다 (e.g. 1e-6)

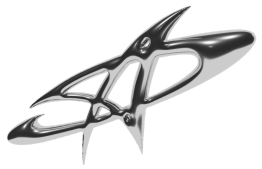
- 한 걸음이 너무 작아 수렴까지 너무 오래 걸림
- 제한된 epoch 안에 loss가 거의 안 줄어듦
- 체감: 학습이 “멈춘 것처럼” 느껴진다



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

코딩 세션

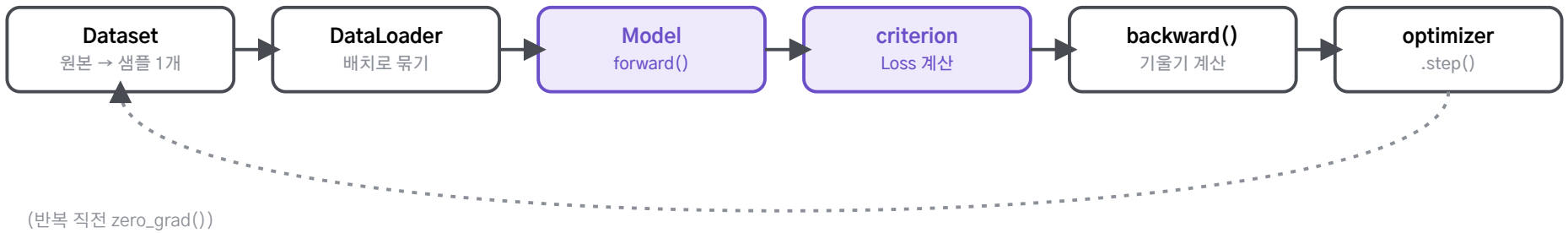
PyTorch 핵심 문법 리마인드 + 첫 학습 루프 직접 짜기



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

오늘 코드로 만들 것의 큰 그림

PyTorch 학습 로직의 표준 구조



다음 배치로 반복 — 이 한 바퀴 × 배치 수 = 1 epoch

앞으로 만들 코드는 전부 이 사이클입니다 — 데이터가 Dataset/DataLoader를 거쳐 Model로, 그 출력이 Loss로, 다시 backward/optimizer로 돌아옵니다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

requires_grad와 backward()가 하는 일

PyTorch의 핵심은 “텐서 연산을 기록해두었다가, backward() 한 번에 모든 기울기를 자동으로 계산”해주는 것입니다.

01_autograd_recap.py

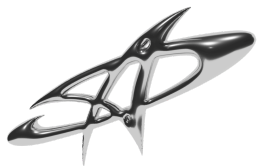
```
import torch

x = torch.tensor([2.0], requires_grad=True)
w = torch.tensor([3.0], requires_grad=True)
b = torch.tensor([1.0], requires_grad=True)

y = w * x + b                 # 순전파(forward)
loss = (y - 5.0) ** 2         # 예: 정답 5와의 차이 제곱

loss.backward()               # 역전파: dL/dw, dL/db 자동 계산
print(w.grad, b.grad)         # 계산된 기울기 확인
```

기억할 것: forward는 우리가 직접 짜지만, backward의 미분 계산은 PyTorch가 대신 해줍니다. 우리가 직접 짜는 건 “언제 zero_grad / backward / step을 부르느냐”는 학습 루프의 흐름입니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

작은 MLP(다층 퍼셉트론) 하나 만들기

02_model.py

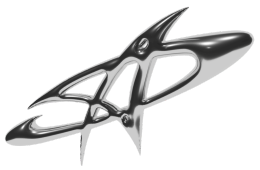
```
import torch.nn as nn

class TinyMLP(nn.Module):
    def __init__(self, in_dim=784, hidden=128, out_dim=10):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden)
        self.act = nn.ReLU()
        self.fc2 = nn.Linear(hidden, out_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.act(x)
        x = self.fc2(x)     # 아직 softmax 안 함 (loss에서 처리)
        return x

model = TinyMLP()
```

nn.Module을 상속하면 **__init__**에서 레이어를 등록하고, **forward**에서 데이터가 흐르는 순서만 정의하면 됩니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

데이터를 배치(batch) 단위로 흘려보내기

03_data.py

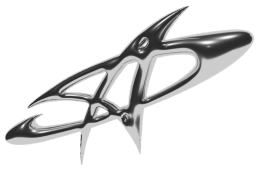
```python
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

tfm = transforms.Compose([transforms.ToTensor()])
train_set = datasets.MNIST(root="./data", train=True,
                           download=True, transform=tfm)

train_loader = DataLoader(train_set, batch_size=64, shuffle=True)

# DataLoader는 (이미지, 정답) 배치를 하나씩 꺼내주는 반복자(iterator)
images, labels = next(iter(train_loader))
print(images.shape, labels.shape)   # [64, 1, 28, 28] [64]
```

Hugging Face의 **datasets** 라이브러리를 쓰면 이 과정이 한 줄로 줄지만, 이번 학기엔 “배치가 실제로 어떻게 만들어지는지”를 직접 눈으로 봅니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

학습 루프 — 이 강의의 핵심 패턴

for epoch: for batch: … (Hugging Face 없이)

04_train_loop.py

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

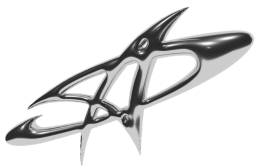
for epoch in range(5):
    for images, labels in train_loader:
        images = images.view(images.size(0), -1)  # [B,1,28,28] → [B,784]

        preds = model(images)                # 1. 순전파
        loss = criterion(preds, labels)      # 2. 손실 계산

        optimizer.zero_grad()                # 3. 이전 기울기 초기화
        loss.backward()                      # 4. 역전파(기울기 계산)
        optimizer.step()                     # 5. 파라미터 업데이트

    print(f"epoch {epoch} loss {loss.item():.4f}")
```

이 5줄(순전파 → 손실 → zero_grad → backward → step)이 **앞으로 13주 내내 반복되는 가장 중요한 패턴**입니다. Transformer가 되어도 이 뼈대는 그대로입니다.



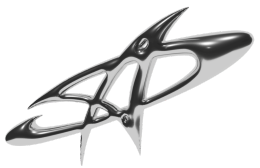
이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

검증(validation)까지 붙이기

학습 중간중간 정확도(accuracy)를 확인하자

- model.eval() + torch.no_grad()로 평가 시에는 기울기 계산을 끔
- 예측값 중 가장 확률이 높은 클래스를 argmax로 뽑아 정답과 비교

```
model.eval()
correct, total = 0, 0
with torch.no_grad():
    for images, labels in test_loader:
        images = images.view(images.size(0), -1)
        preds = model(images).argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
print(f"accuracy: {correct/total:.2%}")
model.train()
```

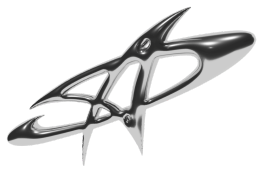


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

실습 환경 체크리스트

다음 주부터 바로 코드를 짤 수 있도록

- **Python 3.10+** 가상환경 (venv 또는 conda) 준비
- **PyTorch** 설치 — GPU가 있다면 CUDA 버전 확인, 없다면 Colab(무료 GPU) 사용 추천
- 설치 확인 명령: **torch.cuda.is_available()** 로 GPU 인식 여부 확인
- 이번 학기 전체 코드는 **Hugging Face의 Trainer/pipeline을 12주차 전까지 사용하지 않습니다**
  — import는 되어 있어도 직접 호출하지 않기



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

오늘 배운 것 & 다음 주 예고

오늘 배운 것

→손실(Loss) = 예측과 정답의 차이를 재는 지표

→경사하강법 = 손실이 줄어드는 방향으로 조금씩 이동

→Optimizer(Adam) = 그 이동을 더 똑똑하게

→forward → loss → zero_grad → backward → step

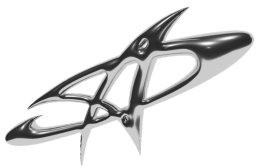
2주차 예고

→RNN/LSTM을 아주 간단히만 훑고 넘어갑니다

→왜 시퀀스 데이터에 기울기 소실 문제가 생기는지

→“그래서 Attention이 필요하다”는 문제의식까지

→코딩: 문자/단어 토크나이저 직접 구현



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지